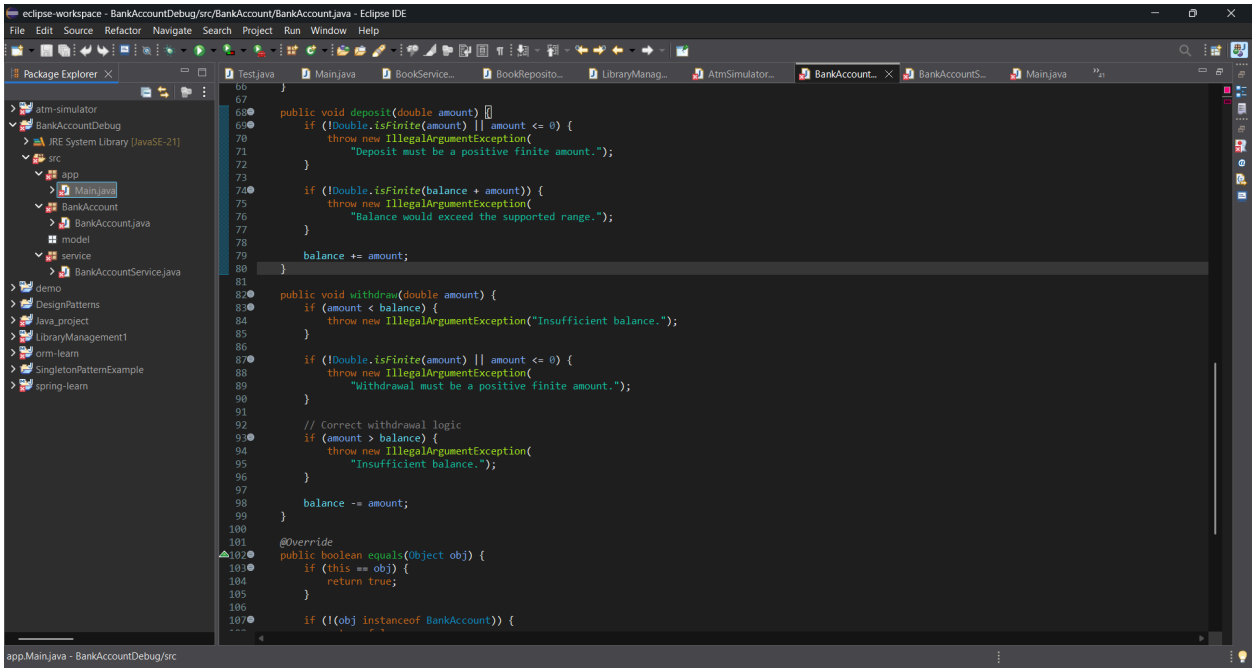
```java
package app;
import model.BankAccount;
import service.BankAccountService;
public class Main {
  public static void main(String[] args) {
    BankAccount account =
        new BankAccount("ACC1001", "Nivashini", 1000.0);
    BankAccountService service = new BankAccountService();
    service.displayBalance(account);
    service.deposit(account, 500.0);
    service.displayBalance(account);
    // Planted bug: attempt to withdraw more than the balance.
    try {
      service.withdraw(account, 2000.0);
    } catch (IllegalArgumentException e) {
      System.out.println("Withdrawal rejected: " + e.getMessage());
    }
    service.displayBalance(account);
    System.out.println(
      "Total accounts created: " + BankAccount.getAccountCounter()
    );
  }
}
```

```java
package service;
import model.BankAccount;
public class BankAccountService {
  public void deposit(BankAccount account, double amount) {
    account.deposit(amount);
    System.out.println("Deposit successful.");
  }
  public void withdraw(BankAccount account, double amount) {
    account.withdraw(amount);
    System.out.println("Withdrawal successful.");
  }
  public void displayBalance(BankAccount account) {
    System.out.printf("Account: %s%n", account.getAccountNumber());
    System.out.printf("Holder: %s%n", account.getHolderName());
    System.out.printf("Balance: %.2f%n", account.getBalance());
  }
}
```

```java
package model;
import java.util.Objects;
public class BankAccount {
```

```
private final String accountNumber;
private String holderName;
private double balance;
private static int accountCounter = 0;
// Constructor 1: no-argument constructor
public BankAccount() {
    this("ACC" + String.format("%04d", accountCounter + 1),
         "Unknown", 0.0);
}
// Constructor 2: account number and holder
public BankAccount(String accountNumber, String holderName) {
    this(accountNumber, holderName, 0.0);
}
// Constructor 3: full constructor
public BankAccount(String accountNumber,
                   String holderName,
                   double balance) {
    if (accountNumber == null || accountNumber.isBlank()) {
        throw new IllegalArgumentException(
            "Account number cannot be empty.");
    }
    if (holderName == null || holderName.isBlank()) {
        throw new IllegalArgumentException(
            "Holder name cannot be empty.");
    }
    if (!Double.isFinite(balance) || balance < 0) {
        throw new IllegalArgumentException(
            "Initial balance must be valid and non-negative.");
    }
    this.accountNumber = accountNumber;
    this.holderName = holderName;
    this.balance = balance;
    accountCounter++;
}
public String getAccountNumber() {
    return accountNumber;
}
public String getHolderName() {
    return holderName;
}
public double getBalance() {
    return balance;
}
public static int getAccountCounter() {
    return accountCounter;
}
public void deposit(double amount) {
    if (!Double.isFinite(amount) || amount <= 0) {
        throw new IllegalArgumentException(
```

```
            "Deposit must be a positive finite amount.");
    }
    if (!Double.isFinite(balance + amount)) {
        throw new IllegalArgumentException(
            "Balance would exceed the supported range.");
    }
    balance += amount;
}
public void withdraw(double amount) {
    if (!Double.isFinite(amount) || amount <= 0) {
        throw new IllegalArgumentException(
            "Withdrawal must be a positive finite amount.");
    }
    // Correct withdrawal logic
    if (amount > balance) {
        throw new IllegalArgumentException(
            "Insufficient balance.");
    }
    balance -= amount;
}
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (!(obj instanceof BankAccount)) {
        return false;
    }
    BankAccount other = (BankAccount) obj;
    return accountNumber.equals(other.accountNumber);
}
@Override
public int hashCode() {
    return Objects.hash(accountNumber);
}
@Override
public String toString() {
    return "BankAccount{accountNumber='" + accountNumber
        + "', holderName='" + holderName
        + "', balance=" + balance + "}";
}
}
```


eclipse-workspace - BankAccountDebug/src/BankAccount/BankAccount.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
atm-simulator
BankAccountDebug
JRE System Library [JavaSE-21]
src
app
Main.java
BankAccount
BankAccount.java
model
service
BankAccountService.java
demo
DesignPatterns
Java_project
LibraryManagement1
orm-learn
SingletonPatternExample
spring-learn
Test.java
Main.java
BookService...
BookReposito...
LibraryManag...
AtmSimulator...
BankAccount...
BankAccountS...
Main.java
66 }
67
68 public void deposit(double amount) {
69 if (!Double.isFinite(amount) || amount <= 0) {
70 throw new IllegalArgumentException(
71 "Deposit must be a positive finite amount.");
72 }
73
74 if (!Double.isFinite(balance + amount)) {
75 throw new IllegalArgumentException(
76 "Balance would exceed the supported range.");
77 }
78
79 balance += amount;
80 }
81
82 public void withdraw(double amount) {
83 if (amount < balance) {
84 throw new IllegalArgumentException("Insufficient balance.");
85 }
86
87 if (!Double.isFinite(amount) || amount <= 0) {
88 throw new IllegalArgumentException(
89 "Withdrawal must be a positive finite amount.");
90 }
91
92 // Correct withdrawal logic
93 if (amount > balance) {
94 throw new IllegalArgumentException(
95 "Insufficient balance.");
96 }
97
98 balance -= amount;
99 }
100
101 @Override
102 public boolean equals(Object obj) {
103 if (this == obj) {
104 return true;
105 }
106
107 if (!(obj instanceof BankAccount)) {
app.Main.java - BankAccountDebug/src